

Syllabus:

NP-Hardness and NP-complete Problems: NP Hardness, NP completeness, Consequences of being in P, Cook's Theorem, Reduction Source Problems, Reductions: Reductions for some known Problems.

Basic Concepts (or) Basic Terminology:

- ✓ Most algorithms we have studied so far have polynomial-time running times - i.e., finding shortest path, sorting of arrays etc. All these algorithms are efficient, because in each case, their time requirement grows as a polynomial function i.e., n, n^2, n^3 of the size of the input.
- ✓ In general, based on the computational complexity the problems are of ② types.

1. Tractable Problems

2. Intractable Problems

1. Tractable Problems:

- ✓ The problems which can be solved in finite amount of time. These are called Polynomial time algorithms.

2. Intractable Problems:

- ✓ The problems which cannot be solved in finite amount of time.
- ✓ Non-Polynomial Time algorithms are comes under Intractable algorithms.

* P , NP , and NP complete.

✓ Somewhere we believe that an efficient algorithm should run in polynomial time, that is $O(n^k)$ for input size n and some constant k .

✓ Consequently, problems that can be solved in polynomial time are treated as "easy problems" and we use P to denote the set of polynomial time solvable problems.

P : The set of problems that can be solved in polynomial time.

Problems solvable at Polynomial Time.

Eg1: Testing whether the given number is prime, finding shortest path etc.

Eg2: Problems that can't solve in Polynomial Time.

1. Factorization, 2. Graph coloring i.e., given k , determine whether a graph has a k vertex coloring.

✓ The problems that cannot be solved in polynomial time so far are called considered as "hard" problems i.e., they can never be solved in polynomial time.

NP : The set of problems for which the "yes" answer (i.e., a solution) can be verified in polynomial time.

NP stands for "Non-deterministic Polynomial Time"

✓ A problem is NP -complete if

1. It is in NP .

2. If it can be solved in polynomial time, then any problem in NP can be solved in polynomial time.

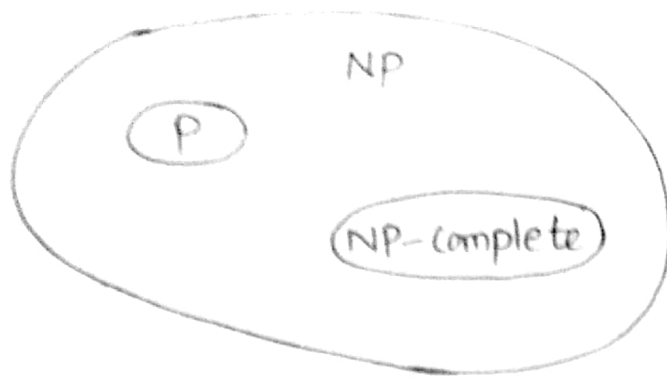


Fig (A): Relationship b/n P, NP & NP-complete

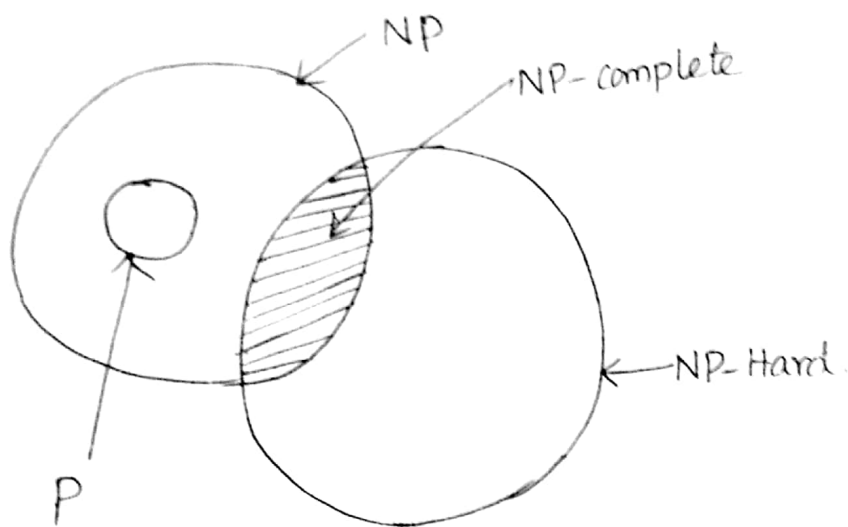


Fig (B): Relationship between P, NP, NP-complete, NP-Hard.

* NP-Hardness:

- ✓ If there is a problem 'x' such that every problem in NP reduces to 'x', then 'x' is at least as hard as every problem in NP. We say that 'x' is hard for NP or NP-Hard.
- ✓ Once we have one problem i.e., NP-hard, we can use it to find others, for example, by reducing x to y to show that y is also NP-Hard.

* NP-completeness:

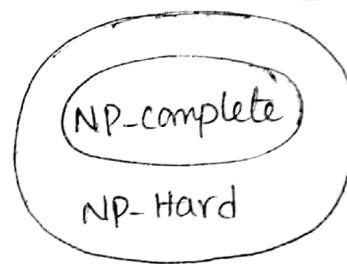
✓ If x' is in NP and is also in NP-hard then we say that x' is complete for NP. or is NP-complete.

✓ If any Problem i.e., NP-hard Problem is in P (has polynomial time alg.) then every problem that reduces to it (which means all of NP), is also in P.

✓ All NP-Complete problems have the same hardness.

✓ If any NP-complete problem is in P, then $P \equiv NP$

$NP\text{-complete} \subseteq NP\text{-Hard}$



* Consequences of being in P:

✓ If we can show that a known NP-hard problem reduces to 'A' in polynomial time, then we know 'A' is NP-hard.
So if 'A' is solvable in polynomial time, then $P \equiv NP$.

✓ we still do not know if $P=NP$. and as ^{from} ~~with~~ research & study by experts we have $P \neq NP$.

* Reduction Source Problems:

✓ To show that our problem 'A' is NP-hard, we can get any known NP-hard problem 'B' to reduce from it.

✓ However, we usually reduce from a known NP-complete problem (NP-hard and also in NP), since if a problem is not known to be in NP, it may be more difficult than our problem.

- ✓ In order to start off the set of NP-hard problems, we need to have one problem that all problems in NP reduce to it.
 - ✓ Then, we can find other NP-hard problems by reducing from it.
- Eg. Circuit Satisfiability Problem.

* Cook's Theorem :

✓ Satisfiability:

"The satisfiability problem is to determine whether a formula is True for some assignment of truth values to the variables".

Eg. CNF satisfiability is the satisfiability problem for CNF formula.

- ✓ S. Cook formulated one question, is there any single problem in NP such that if we showed it in 'P' then that would imply that $P = NP$?
- ✓ Cook answered his own question in his own style with a theorem i.e., Cook's theorem

Statement:

"Satisfiability is in P if and only if $P = NP$."

Proof:

We have already seen that NP is satisfiable.

Hence, if $P = NP$, then satisfiability is in 'P'.

It remains to be shown that, if satisfiability is in P, then $P = NP$. To do this, we need to show that how to obtain from any Polynomial time non-deterministic algorithm A and input I , a formula $\phi(A, I)$ such that ϕ is satisfiable if and only if A is successful termination with

If the length of the input 'I' is 'n' and the time complexities of 'A' is in $P(n)$ and for some polynomial $P(n)$, then the length of 'Q' is

$$O(P^3(n) \log n) \approx O(P^4(n))$$

the time needed to construct 'Q' is also $O(P^3(n) \log n)$.

✓ A deterministic algorithm - 'z' to be determined.

✓ The outcome of 'A' on any input 'I' can be easily obtained.

✓ Algorithm - 'z' simply computes 'Q' and then uses a deterministic algorithm for satisfiability to determine whether 'Q' is satisfiable.

✓ If $O(q(m))$ is the time needed to determine whether a formula of length 'm' is satisfiable. Then, the complexity of 'z' is

$$O(\underbrace{P^3(n) \log n}_{\text{for construction of } Q} + \underbrace{q(P^3(n) \log n)}_{\text{for the verification of } Q})$$

for construction of Q

for the verification of Q.

✓ If satisfiability is in P , then $q(m)$ is a polynomial function of 'm' and the complexity of 'z' becomes $O(r(n))$ for some Polynomial $r(n)$.

✓ Hence, if satisfiability is in P , then for every non-deterministic algorithm 'A', we can obtain a deterministic algorithm - 'z' in P . So, the above construction showed that iff satisfiability is in P then $P = NP$.

Hence, theorem is proved.

This theorem used some assumptions. Some of them are

1. All variables in 'A' are of type integer or boolean.
2. Algorithm - 'A' contains no 'read' and 'write' statements.
3. Algorithm - 'A' contains no constants.
4. Algorithm - 'A' uses simple operations i.e., it contains almost one operation and simple variables.
5. Algorithm - 'A' allowed to contain only following type of statements,
goto, if(condition) then goto 'q', success(), failure()
6. The machine on which 'A' is to be executed is word oriented.

* Reductions:

✓ Reduction is a method by which we show that a particular problem can be solved if another known problem can be solved, by using a kind of mapping to transform the (entries) entities and operations in one problem into corresponding ones in another problem.

✓ Suppose, if a problem 'P' reduces to problem 'Q', if an algorithm for 'Q' can be used to solve 'P' easily. In that case, 'P' is "no harder to solve" than 'Q'.

✓ For example, the problem of deciding, if a number 'n' is prime reduces to the problem of factorizing 'n'.

✓ Any problem requires precise statement (specification). This statement of the problem is given as a formal "language". One way to define reductions is in terms of such language.

Definition of Reduction:

✓ A language 'L₁' reduces to another language L₂ in polynomial time, written as $L_1 \leq_p L_2$, if there exists a polynomial-time (computational) computable function $f: \{0,1\}^* \rightarrow \{0,1\}^*$, such that $x \in L_1$, if and only if $f(x) \in L_2$. In this case, deciding L₁ is no harder than deciding L₂.

Reductions in P:

* If L₁, L₂ are languages and $L_1 \leq_p L_2$, then $L_2 \in P$ implies $L_1 \in P$.

Proof: Let 'f' be the polynomial-time function that reduces L₁ to L₂ and A, a polynomial-time algorithm that decides L₂. Then for any 'x', run A on $y=f(x)$.

If it accepts y , then $x \in L_1$; otherwise, $x \notin L_1$. Since the decision on ' x ' is done in polynomial-time, $L_1 \in P$.

Transitivity^{of} Reductions:

If L_1, L_2 are languages, $L_1 \leq_P L_2$ and $L_2 \leq_P L_3$, then $L_1 \leq_P L_3$.

Proof: If ' f ' and ' g ' are the respective Polynomial-time functions that reduce L_1 to L_2 and L_2 to L_3 , then the composition of ' f ' and ' g ', $g(f(w))$, is a polynomial-time function that reduces L_1 to L_3 .

NP Hardness:

✓ NP-Hardness defines a problem to be at least as difficult or time consuming as any other NP problem. It is defined as: A language ' L ' is NP-hard if $L' \leq_P L$ for every language L' in NP.

NP-completeness:

✓ A language ' L ' is NP-complete (NPC) if it is in NP and NP-hard.

✶ NP-completeness to $NP=P$:

If any NP-complete language can be decided in Polynomial time, then $NP=P$. If any language in NP cannot be decided in Polynomial time, then all NP-complete languages cannot.

✶ Circuit Satisfiability Problem:

Input: A circuit (acyclic) with a single output bit. The decision Problem is: Is there an input assignment that produces an output of 1?

We cannot reduce from each problem in NP individually, so we will do this "in bulk" by describing how any P-checking algorithm can be transformed into a circuit in Polynomial time.

Basic Idea is: Each step of the algorithm can be simulated by a level of the circuit and the number of such levels only will be polynomial.

✓ The input will be the certificate:

$$\text{CIRCUIT-SAT} = \{ \langle C \rangle : C \text{ is a satisfiable Boolean Circuit} \}$$

Here $\langle C \rangle$ is some reasonable encoding of C .

✓ A Boolean circuit is a circuit using AND, OR and NOT gates connected by wires, that have multiple inputs and a single output.

✓ The size of a boolean circuit is the total number of gates and wires used in the circuit.

✓ A circuit is satisfiable if there exists an assignment of truth values (false or true) to its inputs that results in the circuit outputting a true.

CIRCUIT-SAT is NP-complete:

✓ CIRCUIT-SAT is in NP, as for a given circuit one can verify in polynomial time if an assignment yields output '1'. CIRCUIT-SAT is NP-hard, as any NP language 'L' can be reduced to it as follows

✓ Let $A(x, y)$ be a verification algorithm for 'L' that runs in time $T(|x|) = O(|x|^K)$ for some K .

✓ For each step in A , construct a Boolean sub-circuit that maps a configuration (memory state and Program Counter) of a machine that carries A to a next configuration.

Then, hook all these $T(|x|)$ sub-circuits together, making the values of 'y' as the inputs to the entire circuit and the

- single bit as the output that reflects the value of $A(x, y)$.
- Now, the big circuit $C(x)$ is satisfiable (by the value of y) iff x is in the language L . Moreover, the size of C is polynomial in $|x|$, and the transformation of x to C can be done in polynomial time.

* Reductions in NPC

- Let L and L' be two languages and $L' \in \text{NP-complete}$.
- If $L' \leq_p L$, then L is NP-hard.
- In addition, if L is in NP, then L is NP-complete.
- The languages can be proved to be NP-complete through reductions to known NP-complete ones.

* Steps for Proving NP-completeness:

- ✓ Prove that L is in NP.
- ✓ Select an NP-complete language L' .
- ✓ Construct a function f that maps members of L' to members of L .
- ✓ show that $x \in L'$ if and only if $f(x) \in L$.
- ✓ show that f can be computed in polynomial time.

Reductions for some known Problems:

Here, the reductions for some known NP-complete problems like CIRCUIT-SAT, 3-CNF SAT, clique, Vertex cover, Subset-sum, Hamiltonian cycle (HC), Travelling Salesman (TSP) are given below

* Propositional Formulae:

Propositional formulae are defined recursively as follows:

- ✓ Truth-valued variables $x, y, \dots$ are formulae.
- ✓ If P and Q are formulae, then $P \text{ AND } Q$, $P \text{ OR } Q$, $\text{NOT } P$, $P \text{ IMPLIES } Q$, $P \text{ IFF } Q$ are formulae; and.
- ✓ If P is a formula, then (P) is also a formula.

For example,

$$P = ((x_1 \Rightarrow x_2) \vee \neg ((\neg x_1 \Rightarrow x_3) \vee x_4)) \wedge \neg x_2$$

Satisfiability of Propositional Formulae:

- ✓ An assignment to a propositional formula ' P ' is a specification of truth values of its variables.
- ✓ An assignment to ' P ' is satisfying if it makes ' P ' true. A propositional formula ' P ' is satisfiable if there is a satisfying assignment to it.

Eg:

$$P = ((x_1 \Rightarrow x_2) \vee \neg ((\neg x_1 \Leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$

is satisfiable, as $(x_1, x_2, x_3, x_4) = (F, F, F, T)$ is a satisfying assignment

* SAT Problem:

- ✓ Acyclic circuits correspond to boolean formulae, so the satisfiability problem (SAT) of determining if there is a truth assignment for the variables of a boolean formula that produces a result true, is also NP-hard.

There are theorems which have proven that this problem remains NP-hard even in the special case when the formula is expressed as a conjunction of 3-literal disjunctions (3SAT).

These problems are also NP-complete since they are in NP.

Given a propositional formula P , is it satisfiable?

$SAT = \{ \langle P \rangle : P \text{ is a satisfiable propositional formula} \}$

Here $\langle P \rangle$ is some reasonable encoding of P .

Properties of SAT:

SAT is in NP.

We can show this as: for any formula x ; use a problem satisfying assignment as certificate y .

CIRCUIT-SAT reduces to SAT - the reduction function f is defined as follows:

1. Name each wire coming out of a gate with a new variable x_{out} .

2. Introduce a clause for each gate of k inputs, x_{out} iff $(x_{in1} \wedge \dots \wedge x_{ink})$

3. Take the AND of all these clauses together with x_{out} , for example,

$$x_{out} \wedge (x_{out} \iff (x_{in1} \wedge \dots \wedge x_{ink}))$$

We have to show that if CIRCUIT-SAT is NP-complete then SAT is also NP-complete.

✓ A circuit C is satisfiable iff the formula $f(C)$ is satisfiable. If C is satisfiable, then $\exists$ an assignment to its input wires that results in the output being 1.

Therefore, $\exists$ an assignment to all intermediate inputs and output wires, resulting in a satisfying assignment to circuit $f(C)$. If $f(C)$ is satisfiable, then any satisfying assignment

results in a satisfying assignment to the circuit C , by assigning it to variables for the input wires.

✓ The reduction function 'f' can be computed in polynomial time. This can be done by performing a breadth-first search on the graph representation of the circuit generating a clause for each gate.

Hence SAT is NP-complete.

★ 3-Conjunctive Normal Form (3-CNF)-SAT Problem:

✓ A propositional formula is in 3-CNF if it consists of clauses connected by ANDs and each clause is the OR of three literals.

✓ A literal is either a variable or its negation.

$$(x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_3 \vee x_4 \vee \neg x_5)$$

3-CNF-SAT = { <P> : P is a satisfiable 3-CNF propositional formula }

NP-Completeness of 3-CNF-SAT:

3-CNF-SAT is in NP, since 3-CNF SAT and SAT is in NP. SAT reduces to 3-CNF SAT.

The reduction function f:

for any propositional formula P,

1. Fully Parenthesize 'P', so that no connective has more than two arguments.
 2. Parse 'P' into a binary tree with literals as leaves and internal nodes as binary connectives.
 3. Introduce a new variable 'y' for each internal node with connective 'q' and construct a clause $(y/z_1/z_2)$, where z_1 and z_2 are its child literals.
 4. Take ANDs of all these constructed clauses together with the variable introduced for P.
- clause into the conjunctive normal form of

the resulting formula is $f(P)$.

We note that ' P ' is satisfiable iff $f(P)$ is satisfiable, and f is computed in Polynomial time. Hence 3-CNF SAT is NP-complete.

Definition of Graph

A graph $G = (V, E)$ is said to have a clique of size K , if there is a subset V' of V with $|V'| = K$ such that for all $u, v \in V'$, the edge $(u, v) \in E$. In other words, a clique of size K is a complete subgraph of G that consists of a subset of ' K ' vertices.

$CLIQUE = \{ \langle G, K \rangle : \text{graph } G \text{ has a clique of size } K \}$

clique helps to solve the following optimization problem: Given a graph G , find a clique of maximum size.

clique is in NP. We can show that by taking a certificate y that is a set V' of vertices, check if V' is indeed a subset of K vertices of G , check (in $O(K^2)$) if each pair (u, v) in V' has an edge in G .

Further, 3-CNF-SAT reduces to clique.

The reduction function f : for any 3-CNF formula P .

1. create a vertex for each literal, so that $|V| = 3n$

2. for any two literals x and y , create an edge (x, y) if they are not in the clause and not negation of each other.

✓ P is satisfiable iff $f(P) = G(V, E)$ has a clique of size ' n '

1. If ' P ' is satisfiable, then each clause has at least one literal with value 1. collect one such literal from each clause to form a set ' V' ' of corresponding vertices in G . Then V' is a clique of size ' n ' of G .

2. If G has a clique V' of size ' n ', then V' contains exactly one vertex corresponding to one literal in each clause.

Letting these literals have truth-value '1' gives a satisfying assignment of 'P'.

Note that 'f' can be computed in Polynomial time.

* Vertex Cover (VC):

✓ We are given a graph $G = (V, E)$. A vertex cover in this graph is $V' \subseteq V$, such that if an edge $(u, v) \in E$, then $u \in V'$ or $v \in V'$ or both. Note that VC covers all the edges in 'E'.

Problem: Find the VC of minimum size on G .

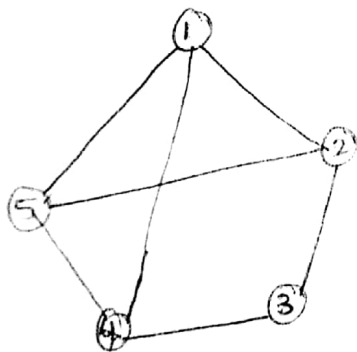
Decision Problem: Does a given G have a VC of a given size k ?

claim - VC is in NPC.

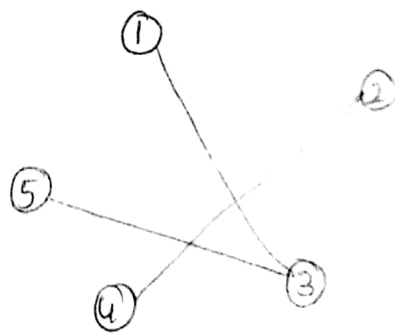
We shall first show that VC is in NP. Given an instance of the problem, that is G, k and subset V' . First verify that $|V'| = k$. Then for each edge (u, v) verify that $u \in V'$ or $v \in V'$. This can be done in Polynomial time. Thus VC is in NP.

To show that VC is NP-hard, we shall reduce clique to VC, i.e., show that

$$\text{Clique} \leq_p \text{VC}$$



Graph G_1



Complement graph G_c

Fig: Example graph & its Complement

The Complement graph G_c contains exactly those edges not in ' G_1 '. This graph can be computed in P-time.

Hamiltonian Cycle (HC):

A Hamiltonian Cycle of a graph G is a cycle which includes every node of G exactly once.

Problem: Hamiltonian Cycle.

Input: $G = (V, E)$

Question: Does G have a Hamiltonian Cycle?

Answer: HC is in NP.

It is easy to show that $HC \in NP$. Given a graph $G \in G$ and a certificate in the form of a sequence of edges, it can be verified that each node is traversed exactly once, in P -time. Hence $HC \in NP$.

To show that HC is NP-hard, we may use a reduction from VC, that is,

$$VC \leq_p HC.$$

Traveling Salesman Problem:

TSP = $\{ \langle G, C, k \rangle : G \text{ is a complete graph, } C \text{ is a non-negative integer-valued cost function on edges of } G, k \text{ is a non-negative integer and } G \text{ has a Hamiltonian cycle with cost less than } k \}$.

$$VC \leq_p TSP \leq_p HC \leq_p TSP$$

It is easy to show that $TSP \in NP$. To show that TSP is NP-hard, we use a reduction from HC as follows:

Given an input $I = (G = (V, E))$ to HC, find an input $I' = (G', C', k')$ to TSP (C' is the cost function) such that G has a HC iff G' has a tour of total cost at most k' .

Let G' be the complete graph on V , $C'(e) = 1$ for each edge $e \in E$, $C'(e) = 2$ for each edge $e \notin E$, $k' = n$, where n

n is the number of nodes in G .

Thus $HC \leq_p TSP$ and

$TSP \in NPC$.

* Independent set:

Def: If $G = (V, E)$ and $V' \subseteq V$, then V' is an independent set iff no two nodes in V' are connected by an edge in E .

Input $G = (V, E), K \in \mathbb{N}, K \leq |V|$

Question: Does ' G ' have an independent set of size at least K ?

Claim: $I \in 3SAT$ iff $(G_1, K_1) \in IS$.

Proof: If ' I ' has a satisfying truth assignment t , then ' G_1 ' has an independent set of size m . To see this, select one literal from each clause made true by t . Two complementary literals cannot be simultaneously made true by t , so the result is an independent set.

Conversely, if G_1 has an independent set V' of size m , then ' I ' has a satisfying truth assignment ~~(truth)~~ t . To see this, observe that V' selects one occurrence of a literal from each clause. Define t to make all of these literals true. Note that for each L , we cannot select both ' L ' and ' $\neg L$ ' in different clauses as they are connected by an edge.

2. Previous Papers (upto 109)

1a) Give description about the classes of NP hard and NP-complete.

b) Explain in detail about the decision problem.

2 Explain the classes of NP-hard and NP-complete.

3a) Show that the Hamiltonian-Path problem is NP-complete.

b) Show that the classes P, viewed as a set of languages, is closed under union, intersection, concatenation, complement, and Kleene star.

That is, if $L_1, L_2 \in P$, then $L_1 \cup L_2 \in P$, etc..

4a) Discuss the Cook's theorem with suitable example.

b)

feasible solution.

✓ A solution which satisfies all linear and non-linear constraints is called Feasible Solution.